PROJECT-COREBANK STUDY GUIDE

내 코드로 공부하는 코어뱅킹 백엔드

지금까지 만든 project-corebank 의 실제 코드를 따라가며 Spring Boot, **JPA**, 트랜잭션, 동시성, 보안, 배치, DB 설계, 테스트, 배포를 정리했다. "이 코드가 왜 이렇게 생겼는지" 를 스스로 설명할 수 있게 되는 것이 목표다.

Java 25 · Spring Boot 4.1 · Hibernate 7.4 (JPA 3.2) · Spring Data JPA · Spring Security · JWT · Spring Batch 6 · Oracle 23ai · Flyway · Testcontainers · Docker · GitHub Actions · React · TypeScript

---

2026-11-10 기준 · 백엔드 main 268 테스트 · 마이그레이션 V1~V13 + R · 저장소 github.com/Csj930205/project-corebank
코드 조각은 실제 파일에서 가져왔고, 길면 주석·import 를 줄였다. 파일 경로는 `src/main/java/com/corebank/` 아래 기준.

이 문서 읽는 법

- **코드를 옆에 열어 두고 읽는다.** 각 절에 파일 이름을 적었다. 문서의 코드는 요약본이고, 주석까지 있는 원본이 더 자세하다.
- **2부(JPA)를 가장 먼저, 천천히.** 이 프로젝트에서 "AI 가 써 줘서 동작은 하지만 이유를 설명하기 어려운" 부분이 대부분 JPA 와 트랜잭션이다.
- 상자 종류: 핵심 꼭 기억할 것 · 함정 자주 하는 실수 · 생각해 보기 답을 먼저 스스로 말해 볼 것.
- **5부의 실습을 직접 해 본다.** 코드를 한 줄 바꾸고 테스트가 어떻게 깨지는지 보는 것이 읽기보다 훨씬 오래 남는다. 끝나면 `git checkout .` 으로 되돌리면 된다.

목차

1부 큰 그림

1. 프로젝트 한눈에 보기
2. 요청 한 건의 여정: 입금
3. 계층 구조와 스프링 기본기

2부 JPA 집중

4. JPA, Hibernate, Spring Data JPA
5. 엔티티 매핑 한 줄씩 읽기
6. 영속성 컨텍스트와 변경 감지
7. save, flush, saveAndFlush
8. 쿼리를 만드는 세 가지 방법
9. 연관관계를 일부러 안 쓴 이유와 N+1
10. 락: 비관적 락과 낙관적 락
11. equals·hashCode 와 프록시
12. 트랜잭션 전파와 롤백
13. JPA 와 JdbcTemplate 섞어 쓰기
14. 설정: validate, open-in-view
15. JPA 함정 모음

3부 도메인과 설계

16. 복식부기 원장
17. 멱등성 키
18. 이체와 데드락
19. 인증·인가 (JWT)
20. 감사 로그와 횟수 제한
21. 일 마감 배치 (Spring Batch)
22. DB 설계 (Oracle, Flyway, 권한)

4부 품질과 운영

23. 테스트 전략
24. 배포 자동화
25. 화면(React)과의 약속

5부 스스로 점검

26. 면접형 질문 40
27. 손으로 해 보는 실습 12
28. 추천 공부 순서

PART 1

큰 그림

무엇을 만들었고, 요청 하나가 어떤 길로 지나가는가

1 프로젝트 한눈에 보기

은행의 핵심 업무(계좌, 입출금, 이체, 원장, 일 마감)를 작게 재현한 API 서버다. 기능 목록보다 중요한 것은 **돈이 틀리지 않게 만든 장치들**이다.

돈이 틀리는 상황	막는 장치	어디서 보나
동시에 두 번 출금	비관적 락(SELECT … FOR UPDATE)으로 같은 계좌의 거래를 한 줄로 세움	10장
네트워크 재시도로 두 번 입금	멱등성 키: 같은 키면 처음 결과를 돌려줌	17장
잔액만 바뀌고 기록은 없음	잔액 변경 + 원장 기록 + 키 기록을 한 트랜잭션으로	6, 12장
A→B, B→A 이체가 서로 기다림	항상 id 작은 계좌부터 잠금	18장
누가 원장을 몰래 고침	DB 권한 + 트리거 + 매일 대사 배치	21, 22장
음수 잔액, 해지 계좌에 잔액	코드 검사 + DB CHECK 제약	22장

패키지 지도

패키지	역할	대표 클래스
`account`	계좌 개설·조회·입출금·해지·거래내역	Account, AccountService, AccountRepository
`ledger`	원장(거래·분개) 기록과 조회	LedgerTransaction, LedgerEntry, LedgerService
`transfer`	계좌 간 이체	TransferService, Transfer
`idempotency`	멱등성 키 확인·기록	IdempotencyService, IdempotencyRecord
`auth` · `customer` · `admin`	로그인, JWT, 고객·관리자, 관리자 조회	SecurityConfig, AuthService, LoginThrottle
`audit` · `ratelimit`	감사 로그, 횟수 제한	AuditLogFilter, RateLimiter
`closing`	일 마감 대사 배치	DailyClosingJobConfig, ClosingService
`common` · `migration`	공통 에러, 시각, 마이그레이션 실행	ApiExceptionHandler, DbTime, MigrationApplication

패키지를 "controller / service / repository" 처럼 계층으로 나누지 않고 **업무(기능)별**로 나눴다. 계좌 기능을 고칠 때 한 폴더만 보면 된다.

2 요청 한 건의 여정: 입금

`POST /accounts/7/deposits`, 헤더 `Authorization: Bearer …`, `Idempotency-Key: 4f1c…`, 본문 `{"amount":10000}` 이 서버에 도착해서 응답이 나갈 때까지를 따라가 보자. 이 흐름을 말로 설명할 수 있으면 프로젝트의 70% 를 이해한 것이다.

AuditLogFilter	감사 대상 요청인지 확인, `X-Request-Id` 발급, 시작 시각 기록. 요청이 끝나면(finally) 결과를 audit_log 에 한 줄 쓴다.
BearerToken 필터	JWT 서명·만료·발급자 확인. 실패하면 컨트롤러까지 가지 않고 401.
Authorization	`/accounts/**` 는 SCOPE_ADMIN 또는 SCOPE_CUSTOMER 필요. 아니면 403.
AccountController	`@Valid` 로 본문 검증(금액 1원 이상), 멱등성 키 형식 검사, `AccountAccessPolicy` 로 "관리자거나 본인 계좌인가" 확인(남의 계좌면 404).
@Transactional 시작	`AccountService.deposit` 진입. 프록시가 DB 커넥션을 잡고 트랜잭션을 연다. 영속성 컨텍스트도 이때 생긴다.
계좌 잠금	`SELECT … FROM account WHERE id=7 FOR UPDATE`. 같은 계좌의 다른 거래는 여기서 기다린다. 조회된 Account 는 영속 상태가 된다.
멱등성 확인	같은 키로 끝난 요청이 있으면 처음 거래를 그대로 돌려주고 끝(잔액 안 바뀜). 다른 내용이면 422.
Account.deposit	자바 객체의 balance 필드만 바뀐다. 아직 SQL 은 없다.
원장 기록	LedgerTransaction 저장(시퀀스로 id 먼저 받음) + 분개 2줄(현금 차변 / 고객예금 대변).
키 기록	`saveAndFlush` 로 idempotency_record 를 즉시 INSERT. 유니크 제약 위반이면 이 자리에서 예외.
커밋	flush: 변경 감지로 `UPDATE account SET balance=…` 와 남은 INSERT 실행 → COMMIT → 잠금 해제. 중간 어디서든 예외면 전부 ROLLBACK.
응답	201 Created + 거래 정보(DTO). 재전송이면 `Idempotent-Replayed: true`. 마지막으로 감사 로그 한 줄.

> **핵심**
> 필터(보안·감사) → 컨트롤러(HTTP·검증·권한) → 서비스(**트랜잭션 경계**, 업무 순서) → 엔티티(**규칙**) → 리포지토리(DB). 서비스 메서드 하나가 트랜잭션 하나이고, 그 안의 모든 DB 작업은 함께 커밋되거나 함께 롤백된다.

3 계층 구조와 스프링 기본기

생성자 주입

모든 서비스는 필드가 `final` 이고 생성자로 의존성을 받는다. `@Autowired` 필드 주입을 쓰지 않는 이유: 테스트에서 직접 `new` 할 수 있고, 빠진 의존성이 컴파일·시작 시점에 드러나며, 객체가 만들어진 뒤 바뀌지 않는다.

transfer/TransferService.java

```
@Service
public class TransferService {
    private final AccountRepository accountRepository;
    private final TransferRepository transferRepository;
    private final LedgerService ledgerService;
    private final IdempotencyService idempotencyService;

    public TransferService(AccountRepository accountRepository, TransferRepository transferRepository,
            LedgerService ledgerService, IdempotencyService idempotencyService) { ... }
```

DTO 는 record, 엔티티는 밖으로 내보내지 않는다

요청·응답은 `record` (`DepositRequest`, `AccountResponse`)다. 엔티티를 JSON 으로 바로 내보내지 않는 이유는 (1) 테이블 구조가 API 계약이 되어 버리고 (2) 지연 로딩 필드가 직렬화 중에 쿼리를 날리거나 예외를 내며 (3) 내보내면 안 되는 값(비밀번호 해시)이 섞일 수 있어서다. 응답 DTO 는 `AccountResponse.from(account)` 같은 정적 메서드로 만든다.

예외는 한곳에서 HTTP 로 바꾼다

서비스는 업무 예외(`InsufficientBalanceException`)만 던지고, `@RestControllerAdvice` 인 `ApiExceptionHandler` 가 상태 코드와 `{code, message}` 로 바꾼다. 서비스는 HTTP 를 모른다.

common/ApiExceptionHandler.java

```
@ExceptionHandler(InsufficientBalanceException.class)
@ResponseStatus(HttpStatus.UNPROCESSABLE_CONTENT)   // 422: 형식은 맞지만 업무 규칙상 불가
public ErrorResponse handleInsufficientBalance(InsufficientBalanceException e) {
    return new ErrorResponse("INSUFFICIENT_BALANCE", e.getMessage());
}
```

상태 코드	이 프로젝트에서의 의미	예
400	요청 자체가 잘못됨	금액 0, 같은 계좌로 이체, 날짜 형식 오류
401 / 403	로그인 필요 / 권한 없음	토큰 없음 / 고객이 /admin 접근
404	없음 (또는 남의 것이라 숨김)	없는 계좌, 남의 계좌
409	현재 상태와 충돌	해지 계좌에 입금, 같은 날 마감 실행 중
422	업무 규칙 위반	잔액 부족, 멱등성 키 재사용
429	횟수 초과 (+ Retry-After)	로그인 잠금, 수취인 조회 한도

시각은 Clock 빈으로

`LocalDateTime.now()` 를 직접 부르면 테스트에서 "어제", "30분 뒤" 를 만들 수 없다. 그래서 `Clock` 을 빈으로 두고 (common/ClockConfig) 주입받는다. 테스트는 고정 시계로 바꿔 끼워 만료 토큰·과거 날짜 마감을 검증한다.

PART 2

JPA 집중

save 를 안 불렀는데 왜 저장되는가 — 영속성 컨텍스트부터 락까지

---

4 JPA, Hibernate, Spring Data JPA

세 이름이 섞여서 헷갈리기 쉽다. 층이 다르다.

이름	정체	이 프로젝트에서 보이는 곳
JPA (Jakarta Persistence 3.2)	표준 **명세**(인터페이스와 애너테이션). "객체를 테이블에 이렇게 매핑한다" 는 약속	`jakarta.persistence.*`: @Entity, @Id, @Column, EntityManager, LockModeType
Hibernate 7.4	JPA 의 **구현체**. 실제로 SQL 을 만들고 변경 감지·캐시를 한다	로그의 SQL, `ddl-auto=validate`, ORA 오류를 예외로 바꾸는 부분
Spring Data JPA	JPA 위의 **편의 계층**. 인터페이스만 만들면 구현을 자동 생성	`AccountRepository extends JpaRepository`, 메서드 이름 쿼리, Page

ORM 이 해 주는 일: 테이블 행 ↔ 자바 객체 변환, 객체 필드 변경을 감지해 UPDATE 만들기, 같은 트랜잭션 안에서 같은 행은 같은 객체로 돌려주기(1차 캐시), 지연 로딩. **대신 알아야 할 것**: 언제 SQL 이 나가는지(flush), 언제 객체가 DB 와 연결이 끊기는지(준영속), 연관관계가 몇 번의 쿼리를 만드는지(N+1).

> **생각해 보기**
> 입금 메서드 `AccountService.deposit` 에는 `accountRepository.save(account)` 가 없다. 그런데 잔액이 DB 에 저장된다. 왜일까? → 6장

5 엔티티 매핑 한 줄씩 읽기

account/Account.java (요약)

```
@Getter                      // Lombok: getter 만. setter 는 일부러 만들지 않는다
@Entity                      // JPA 가 관리하는 객체라는 표시
@Table(name = "account")     // 매핑할 테이블
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "account_seq_generator")
    @SequenceGenerator(name = "account_seq_generator", sequenceName = "account_seq", allocationSize =
1)
    private Long id;                                  // Long(객체): 저장 전에는 null

    @Column(name = "account_number", nullable = false, updatable = false, length = 13)
    private String accountNumber;

    @Column(name = "owner_name", nullable = false, updatable = false, length = 100)
    private String ownerName;

    @Column(nullable = false)
    private long balance;                             // 원 단위 정수. double 금지

    @Enumerated(EnumType.STRING)                      // "ACTIVE" 문자열로 저장
    @Column(nullable = false, length = 10)
    private AccountStatus status;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "customer_id", updatable = false)
    private Long customerId;                          // 연관관계 대신 id 값만

    @Column(name = "closed_at")
    private LocalDateTime closedAt;

    protected Account() { }                           // JPA 전용 기본 생성자

    public static Account open(String accountNumber, String ownerName) { ... }  // 정적 팩토리
    public void deposit(long amount) { ... }          // 의미 있는 변경 메서드
    public void withdraw(long amount) { ... }
    public long close() { ... }
}
```

코드	왜 이렇게 했나
`GenerationType.SEQUENCE`	Oracle 은 시퀀스가 기본 도구다. IDENTITY(자동 증가 열)와 달리 **INSERT 전에 id 를 미리 받을 수 있어** 저장 직후 `getId()` 를 쓸 수 있고, INSERT 를 커밋 때로 미룰 수 있다(7장).
`allocationSize = 1`	Hibernate 기본값은 50 이다(시퀀스 한 번에 50개 번호를 메모리에 받아 둠). 이 값은 DB 시퀀스의 `INCREMENT BY` 와 같아야 한다. V1 에서 `INCREMENT BY 1` 로 만들었으므로 1. 대신 저장할 때마다 NEXTVAL 왕복이 한 번 생긴다(성능과 단순함의 맞바꿈).
`nullable=false, length`	테이블 정의를 문서처럼 보여 주고, `ddl-auto=validate` 가 테이블과 맞는지 시작할 때 검사한다. 실제 제약은 Flyway SQL 이 만든다.
`updatable = false`	이 열은 Hibernate 가 만드는 UPDATE 문에서 빠진다. 계좌번호·이름·개설일은 한 번 정하면 바뀌지 않는다. DB 계정 분리 후 앱 계정은 balance/status/closed_at 열만 UPDATE 권한이 있어서, 이게 빠지면 잔액 변경마다 권한 오류가 난다(22장).
`EnumType.STRING`	기본값 ORDINAL 은 0, 1 같은 순번을 저장한다. enum 중간에 값을 추가하면 기존 데이터의 의미가 바뀌는 사고가 난다. STRING 은 이름으로 저장해 안전하다.
`long balance`	double 은 0.1+0.2 가 정확히 0.3 이 아니다. 원화는 정수 원 단위라 long 으로 충분하다. 넘침은 `Math.addExact` 로 예외를 낸다.
`protected Account()`	JPA(Hibernate)는 조회 결과로 객체를 만들 때 기본 생성자를 리플렉션으로 부른다. 지연 로딩 프록시가 상속할 수 있도록 private 가 아니라 protected. 애플리케이션 코드는 쓰지 못하게 public 도 아니다.
setter 없음 + 정적 팩토리	`Account.open()` 으로만 만들어지니 모든 계좌는 "잔액 0, ACTIVE" 로 시작한다. 잔액은 `deposit/withdraw/close` 로만 바뀌고, 그 안에 규칙(해지 계좌 거부, 잔액 부족 거부)이 있다. "아무나 setBalance" 를 막는 것이 엔티티 설계의 핵심이다.

> **핵심: 규칙은 엔티티에, 순서는 서비스에**
>
> `Account.withdraw` 는 "해지 계좌면 거부, 잔액보다 크면 거부" 라는 **규칙**을 안다. `AccountService.withdraw` 는 "잠그고 → 키 확인 → 출금 → 원장 → 키 기록" 이라는 **순서와 트랜잭션**을 안다. 엔티티는 원장을 모르고(close 는 지급액만 돌려준다), 서비스는 잔액 계산을 모른다.

원장 엔티티는 모든 열이 updatable = false

`LedgerTransaction`, `LedgerEntry` 는 한 번 쓰면 절대 바뀌지 않는 기록이라 모든 열이 `updatable = false` 이고 변경 메서드가 없다. 생성자는 패키지 전용 정적 메서드(`LedgerTransaction.deposit(…)`)뿐이라 `ledger` 패키지 밖에서는 원장 행을 만들 수도 없다. 오직 `LedgerService` 를 거쳐야 한다.

6 영속성 컨텍스트와 변경 감지

영속성 컨텍스트는 트랜잭션마다 생기는 "엔티티 보관함" 이다(Hibernate 의 Session, JPA 의 EntityManager 가 관리). 이 보관함에 들어 있는 엔티티를 **영속(managed) 상태**라고 한다.

상태	뜻	이 프로젝트에서
비영속 (new)	new 로 만들기만 함. JPA 가 모름	`Account.open(…)` 직후
영속 (managed)	보관함에 있음. 변경을 감시함	`save()` 후, `findById` / `findByIdForUpdate` 로 읽은 직후
준영속 (detached)	트랜잭션이 끝나 보관함이 닫힘. 고쳐도 반영 안 됨	서비스가 돌려준 Account 를 컨트롤러가 받을 때
삭제 (removed)	삭제 예약	쓰지 않음 (계좌·원장은 지우지 않는다)

save 없이 저장되는 이유: 변경 감지(dirty checking)

account/AccountService.java

```
@Transactional
public IdempotentResult<LedgerTransaction> deposit(Long id, long amount, IdempotencyKey key) {
    Account account = lock(id);          // ① SELECT … FOR UPDATE → 영속 상태 + 스냅숏 저장
    ...
    account.deposit(amount);             // ② 자바 필드만 바꿈. SQL 없음
    LedgerTransaction tx = ledgerService.recordDeposit(id, amount, account.getBalance());
    idempotencyService.record(...);      // saveAndFlush → 이때 flush 가 한 번 일어난다
    return IdempotentResult.created(tx);
}                                        // ③ 커밋 직전 flush: 스냅숏과 비교 → UPDATE account
```

시점	메모리(영속성 컨텍스트)	DB 로 나간 SQL
① 잠금 조회	Account#7 {balance=5000} + 스냅숏 {balance=5000}	SELECT … FOR UPDATE
② deposit	Account#7 {balance=15000}, 스냅숏은 그대로 5000	(없음)
원장 save	LedgerTransaction 영속 (id 는 시퀀스에서 받음)	SELECT ledger_transaction_seq.NEXTVAL
saveAndFlush	flush: 모아 둔 변경을 전부 내보냄	UPDATE account …, INSERT ledger_transaction, INSERT ledger_entry ×2, INSERT idempotency_record
③ 커밋	다시 flush(바뀐 것 없음) → COMMIT → 보관함 닫힘	COMMIT (잠금 해제)

flush 순서는 Hibernate 가 정한다(INSERT → UPDATE → DELETE 순으로 모아 실행하는 것이 기본). 실제 순서는 실습 1 에서 SQL 로그로 직접 보자.

핵심

트랜잭션 안에서 **조회한** 엔티티는 필드만 바꾸면 커밋 때 UPDATE 된다. `save()` 는 **새** 엔티티를 보관함에 넣을 때 필요하다. Hibernate 는 기본적으로 **바뀐 열만이 아니라 updatable 인 모든 열**을 UPDATE 문에 넣는다(`@DynamicUpdate` 를 붙이면 바뀐 열만).

함정: 트랜잭션 밖에서 고치면 저장되지 않는다

컨트롤러에서 `Account a = accountService.get(7); a.deposit(100);` 처럼 하면 아무 일도 일어나지 않는다. `get` 의 트랜잭션이 끝나 `a` 는 준영속이다. 그래서 변경은 반드시 `@Transactional` 서비스 메서드 안에서 한다.

readOnly = true 가 하는 일

조회 메서드에는 `@Transactional(readOnly = true)` 를 붙였다. 스프링은 이때 Hibernate 세션을 읽기 전용으로 두고 flush 모드를 MANUAL 로 바꾼다. 그래서 **스냅숏을 만들지 않고 변경 감지도 하지 않는다**(메모리·CPU 절약). 실수로 엔티티를 고쳐도 UPDATE 가 나가지 않는다.

1차 캐시와 동일성

같은 트랜잭션 안에서 `findById(7)` 를 두 번 부르면 두 번째는 SQL 없이 보관함의 같은 객체를 돌려준다(`a1 == a2`). 이체에서 두 계좌를 잠근 뒤 `from` , `to` 를 고치면, 커밋 때 두 계좌 모두 UPDATE 되는 것도 같은 원리다.

7 save, flush, saveAndFlush

Spring Data 의 save() 는 무엇을 하나

`SimpleJpaRepository.save(entity)` 는 "새 엔티티인가?" 를 보고 갈린다. 기본 판단은 **id 가 null 이면 새 것**.

- 새 것 → `em.persist(entity)` : 보관함에 넣는다. SEQUENCE 전략이면 이때 NEXTVAL 로 id 를 받아 객체에 채운다. INSERT 는 flush 때.
- 아니면 → `em.merge(entity)` : 준영속 객체의 값을 영속 객체에 복사(없으면 SELECT 로 찾아옴). 이 프로젝트는 merge 경로를 쓰지 않는다. 변경은 항상 "조회 → 메서드 호출 → 변경 감지" 다.

ledger/LedgerService.java

```
LedgerTransaction transaction = transactionRepository.save(
        LedgerTransaction.deposit(accountId, amount, balanceAfter));
        // ↑ persist: id 는 확정, INSERT 는 아직
entryRepository.saveAll(List.of(
        LedgerEntry.cashDebit(transaction.getId(), amount),              // 바로 id 를 쓸 수 있다
        LedgerEntry.customerDepositCredit(transaction.getId(), accountId, amount)));
```

> **생각해 보기**
> id 생성을 `GenerationType.IDENTITY` 로 바꾸면 위 코드는 어떻게 동작할까? (IDENTITY 는 INSERT 를 해야 id 를 알 수 있어 persist 순간 INSERT 가 바로 나간다. 그래서 JDBC 배치 INSERT 도 못 쓴다.)

flush 는 언제 일어나나

- 트랜잭션 커밋 직전 (가장 흔함)
- JPQL·네이티브 쿼리를 실행하기 직전, 그 쿼리 결과에 영향을 줄 변경이 있으면 (FlushMode.AUTO). 쿼리가 방금 고친 값을 보도록.
- `flush()` , `saveAndFlush()` 를 직접 부를 때

멱등성 기록이 saveAndFlush 인 이유

idempotency/IdempotencyService.java

```
@Transactional(propagation = Propagation.MANDATORY)
public void record(IdempotencyKey key, IdempotentOperation operation,
                   String fingerprint, Long resourceId) {
    try {
        repository.saveAndFlush(new IdempotencyRecord(key, operation, fingerprint, resourceId));
    } catch (DataIntegrityViolationException e) {        // 유니크 제약(키 중복)
        throw new IdempotencyKeyReusedException(key);    // → 422
    }
}
```

`save()` 만 하면 INSERT 는 커밋 때 나가고, 유니크 제약 위반도 **커밋 때** 터진다. 그때는 이미 이 메서드의 try 블록을 벗어났으니 catch 로 잡을 수 없고, 스프링이 감싼 다른 예외로 500 이 될 수 있다. `saveAndFlush` 로 INSERT 를 **지금** 실행하면 위반이 try 안에서 터져 의미 있는 업무 예외(키 재사용)로 바꿀 수 있다.

> **함정: 예외를 잡아도 트랜잭션은 이미 망가졌을 수 있다**
> JPA 예외가 난 뒤의 영속성 컨텍스트는 믿을 수 없는 상태라, 잡고 나서 계속 진행하면 안 된다. 여기서는 잡자마자 업무 예외로 다시 던져 **전체를 롤백**한다. "잡고 무시하고 계속" 은 JPA 에서 위험한 패턴이다.

8 쿼리를 만드는 세 가지 방법

① 메서드 이름 쿼리

ledger/LedgerTransactionRepository.java, account/AccountRepository.java

```
List<LedgerTransaction> findByAccountIdAndIdLessThanOrderByIdDesc(Long accountId, Long before, Limit
limit);
//  → SELECT … FROM ledger_transaction WHERE account_id = ? AND id < ? ORDER BY id DESC FETCH FIRST ?
ROWS ONLY

boolean existsByIdAndCustomerId(Long id, Long customerId);   // 존재 여부만 (행 전체를 읽지 않음)
Optional<Account> findByAccountNumber(String accountNumber);
```

Spring Data 가 이름을 해석해 JPQL 을 만든다. 조건 2~3개까지는 읽기 좋다. 이름에 쓰는 것은 **테이블 열 이름이 아니라 엔티티 필드 이름**(`accountId`)이다. `Limit` 은 Spring Data 3.2 에 생긴 "몇 건만" 파라미터다.

② JPQL (@Query)

account/AccountRepository.java

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Account a WHERE a.id = :id")
Optional<Account> findByIdForUpdate(@Param("id") Long id);

@Query("""
        SELECT a FROM Account a
        WHERE (:numberPrefix IS NULL OR a.accountNumber LIKE :numberPrefix ESCAPE '\\'
               OR LOWER(a.ownerName) LIKE :namePattern ESCAPE '\\')
          AND (:status IS NULL OR a.status = :status)
          AND (:minBalance IS NULL OR a.balance >= :minBalance) ...
        """)
Page<Account> search(..., Pageable pageable);
```

JPQL 은 **엔티티와 필드**를 대상으로 쓰는 SQL 비슷한 언어다(`FROM Account a` 의 Account 는 클래스 이름). Hibernate 가 Oracle SQL 로 번역한다. `(:x IS NULL OR …)` 는 "값이 없으면 조건을 빼라" 는 선택 조건 패턴이다. 조건이 아주 많아지면 Querydsl 이나 Specification 같은 동적 쿼리 도구를 쓴다.

Page 와 Pageable: 반환형이 `Page` 면 Spring Data 가 목록 쿼리와 함께 **COUNT 쿼리**를 한 번 더 날려 전체 건수를 채운다. 관리자 목록은 "전체 몇 쪽" 이 필요해서 Page 를 쓰고, 고객 거래내역은 COUNT 가 필요 없어 커서 방식(`before` + `size+1` 건 조회)을 쓴다.

③ 네이티브 SQL

account/AccountRepository.java

```
@Query(value = "SELECT account_number_seq.NEXTVAL FROM dual", nativeQuery = true)
long nextAccountNumberSequence();
```

Oracle 전용 문법(`NEXTVAL`, `dual`)은 JPQL 로 쓸 수 없어 네이티브로 쓴다. 집계·윈도 함수가 많은 조회(관리자 요약, 일 마감)는 아예 JdbcTemplate 으로 썼다(13장).

방법	좋은 곳	조심할 곳
메서드 이름	단순 조건 조회	조건이 늘면 이름이 끝없이 길어진다
JPQL	엔티티로 받는 조회, 락, 페이지	DB 전용 함수는 제한적. 문자열이라 오타는 시작 시점에 검증됨
네이티브 / JdbcTemplate	DB 전용 기능, 집계, 대량 처리	엔티티 변경 감지·1차 캐시와 따로 논다(13장)

9 연관관계를 일부러 안 쓴 이유와 N+1

JPA 수업에서 가장 많이 나오는 `@ManyToOne`, `@OneToMany` 가 이 프로젝트에는 **하나도 없다**. 계좌·원장·이체는 서로를 **id 값**(`Long accountId`)으로만 가리킨다.

ledger/LedgerTransaction.java 와 가상의 다른 설계

```
// 이 프로젝트 (id 값만)
@Column(name = "account_id", nullable = false, updatable = false)
private Long accountId;

// 만약 연관관계로 했다면
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "account_id")
private Account account;
```

	id 값만 (이 프로젝트)	@ManyToOne
쿼리 예측	쓴 그대로 나간다	지연 로딩이 언제 쿼리를 날리는지 알아야 한다
결합	원장 패키지는 Account 클래스를 모른다	원장이 계좌 엔티티에 묶인다
무결성	DB 외래키가 지킨다	DB 외래키 + 객체 참조
편의	필요하면 id 로 직접 조회	`tx.getAccount().getOwnerName()` 처럼 탐색 가능

원장은 "계좌 7번에 이 일이 있었다" 는 **기록**이지 계좌 객체의 일부가 아니다. 그래서 값으로 충분하다고 판단했다. 이것이 정답이라는 뜻은 아니다. 애그리거트(함께 바뀌는 묶음) 안에서는 연관관계가 자연스럽고, 애그리거트 사이는 id 로 잇는 것이 DDD 에서 흔한 기준이다.

N+1 문제와 이 프로젝트의 해결

거래내역 20건을 보여 주면서 이체 상대 계좌 이름도 보여 줘야 한다. 거래마다 상대 계좌를 조회하면 **1(목록) + N(건마다 하나)** 번의 쿼리가 나간다. 이것이 N+1 이다. 연관관계 + 지연 로딩에서 `for (tx : list) tx.getAccount().getName()` 할 때 같은 일이 눈에 안 보이게 일어난다.

ledger/LedgerService.findHistory, account/AccountService.history

```
// 1) 이체 id 들을 모아 상대 쪽 거래를 IN 한 번으로
transactionRepository.findByTransferIdInAndAccountIdNot(transferIds, accountId);
// 2) 상대 계좌 id 들을 모아 계좌를 IN 한 번으로
Map<Long, Account> counterparties = accountRepository.findAllById(counterpartyIds).stream()
        .collect(Collectors.toMap(Account::getId, Function.identity()));
// → 건수와 상관없이 쿼리 3번(목록, 상대 거래, 상대 계좌)
```

핵심: N+1 의 해법 세 가지

1. **모아서 IN 조회** (이 프로젝트) — 가장 단순하고 예측 가능
2. **fetch join**: `SELECT t FROM Tx t JOIN FETCH t.account` — 연관관계가 있을 때 한 쿼리로
3. **배치 사이즈**: `hibernate.default_batch_fetch_size=100` — 지연 로딩을 IN 으로 묶어 줌

EAGER(즉시 로딩)는 해법이 아니다. 필요 없을 때도 항상 읽고, JPQL 에서는 오히려 N+1 을 만든다. 연관관계는 기본을 LAZY 로 둔다.

10 락: 비관적 락과 낙관적 락

락이 없으면: 갱신 분실(lost update)

시각	요청 A: 7,000원 출금	요청 B: 7,000원 출금	DB 잔액
t1	잔액 10,000 읽음		10,000
t2		잔액 10,000 읽음	10,000
t3	10,000 ≥ 7,000 통과, 3,000 으로 UPDATE·커밋		3,000
t4		10,000 ≥ 7,000 통과(!), 3,000 으로 UPDATE·커밋	3,000

14,000원이 나갔는데 잔액은 3,000원. 엔티티의 잔액 검사는 **읽은 값**으로 하기 때문에 동시성 앞에서는 무력하다.

비관적 락 (이 프로젝트)

account/AccountRepository.java

```
@Lock(LockModeType.PESSIMISTIC_WRITE)                  // → Oracle: SELECT … FOR UPDATE
@Query("SELECT a FROM Account a WHERE a.id = :id")
Optional<Account> findByIdForUpdate(@Param("id") Long id);
```

B 는 t2 에서 A 가 커밋할 때까지 **기다렸다가** 3,000 을 읽고 잔액 부족으로 거부된다. 락은 트랜잭션이 끝날 때 풀리므로, 락을 건 조회는 반드시 `@Transactional` 안에서 해야 한다(밖에서 부르면 조회 직후 풀려 의미가 없다).

낙관적 락과 비교

가상의 낙관적 락 코드

```
@Version
private Long version;   // UPDATE account SET …, version = 6 WHERE id = 7 AND version = 5
                        // 0건 갱신이면 ObjectOptimisticLockingFailureException
```

	비관적 락 (FOR UPDATE)	낙관적 락 (@Version)
방식	먼저 잠그고 순서대로 처리	잠그지 않고, 커밋 때 버전이 바뀌었으면 실패
충돌 시	기다린다 → 결국 성공 또는 업무 거부	예외 → 앱이 재시도해야 한다
어울리는 곳	충돌이 잦고, 실패하면 안 되는 일 (같은 계좌 동시 거래)	충돌이 드물고 재시도가 쉬운 일 (프로필 수정)
비용	대기, 데드락 가능성(순서로 해결, 18장)	재시도 로직, 사용자에게 "다시 시도" 노출

함정: 락은 기본적으로 무한정 기다린다

Oracle 의 FOR UPDATE 는 기본적으로 상대가 끝날 때까지 기다린다. 운영에서는 `jakarta.persistence.lock.timeout` 힌트로 대기 시간을 제한하는 것을 검토한다. 그리고 락을 쥔 트랜잭션 안에서 외부 API 호출처럼 오래 걸리는 일을 하면 안 된다.

검증: `WithdrawalIntegrationTest.concurrentWithdrawalsNeverOverdraw` — 잔액 10,000 계좌에 3,000원 출금 10건을 스레드 8개로 동시에 보내 정확히 3건 성공, 잔액 1,000, 원장 차대 합 0 을 확인한다. 실습 2 에서 `@Lock` 을 지우고 돌려 보자.

11 equals·hashCode 와 프록시

account/Account.java

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Account other)) return false;          // getClass() 아님: 프록시는 하위 클래스
    return getId() != null && getId().equals(other.getId());  // 필드 아닌 getter: 프록시 필드는 비어 있음
}

@Override
public int hashCode() {
    return Account.class.hashCode();    // 계좌마다 다르지 않은 고정값
}
```

- **id 로만 비교**: 잔액 같은 바뀌는 값으로 비교하면 입금 한 번에 "다른 계좌" 가 되어 Set/Map 에서 못 찾는다.
- **저장 전(id null)**은 자기 자신하고만 같다. 아직 어떤 행인지 정해지지 않았으니까.
- **hashCode 가 고정값인 이유**: id 로 해시를 만들면 저장 전(null)과 저장 후(시퀀스 값)의 해시가 달라 HashSet 에 넣어 둔 객체를 잃어버린다. 같은 버킷에 모여 느려지는 단점은, 계좌를 수백만 개씩 한 Set 에 담지 않으므로 감수한다.
- **프록시**: 지연 로딩(`getReferenceById` , LAZY 연관관계)은 엔티티를 상속한 가짜 객체를 준다. 그 객체의 필드는 비어 있고, getter 를 부를 때 진짜를 읽어 온다. 그래서 instanceof 와 getter 를 쓰고, 클래스를 final 로 두지 않는다.

핵심

Lombok `@EqualsAndHashCode` , `@Data` 를 엔티티에 붙이지 않는다. 모든 필드로 비교하고, 연관관계가 있으면 지연 로딩을 건드리거나 무한 순환을 만든다.

12 트랜잭션 전파와 롤백

`@Transactional` 은 스프링이 만든 **프록시**가 메서드 앞뒤에서 트랜잭션을 열고 닫는 방식이다. 이 사실에서 대부분의 규칙이 나온다.

이 프로젝트에서 쓴 전파 옵션

옵션	뜻	어디서, 왜
REQUIRED (기본)	있으면 참여, 없으면 새로	AccountService, TransferService 의 업무 메서드
MANDATORY	반드시 기존 트랜잭션 안에서. 없으면 예외	LedgerService, IdempotencyService. 원장·키 기록이 **업무와 따로 커밋되면 안 된다**는 것을 코드로 강제
REQUIRES_NEW	기존 것을 잠시 멈추고 새 트랜잭션으로, 바로 커밋	LoginThrottle.acquire, RateLimiter.hit. **업무가 실패해도 시도 횟수는 남아야** 하므로
readOnly = true	읽기 전용 힌트 (6장)	모든 조회 메서드
(트랜잭션 없음)		ClosingService.run: Spring Batch 가 단계마다 스스로 트랜잭션을 연다. 바깥에 열려 있으면 배치가 거부한다

auth/LoginThrottle.java

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void acquire(String username) {   // 로그인 시도를 "실패" 로 미리 세고 바로 커밋
    ...                                  // 비밀번호가 틀려 login() 이 예외로 끝나도 이 기록은 남는다
}
```

롤백 규칙

- 기본: **RuntimeException(언체크)와 Error 는 롤백**, 체크 예외는 커밋. 이 프로젝트의 업무 예외는 모두 RuntimeException 이라 던지면 롤백된다.
- 이체에서 입금 쪽(`to.deposit`)이 실패하면, 이미 실행한 `from.withdraw` 도 커밋되지 않는다. 원장·키 기록도 함께 사라진다.

test: account/DepositRollbackIntegrationTest.java (요약)

```
@MockitoSpyBean
private LedgerEntryRepository entryRepository;   // 진짜 빈을 감싼 스파이

@Test
void balanceAndTransactionRollBackWhenEntriesFail() {
    Long id = accountService.open("롤백").getId();
    doThrow(new IllegalStateException("분개 저장 실패")).when(entryRepository).saveAll(anyList());

    assertThatThrownBy(() -> accountService.deposit(id, 1000, newKey()).value())
            .isInstanceOf(IllegalStateException.class);

    assertThat(accountService.get(id).getBalance()).isZero();              // 잔액도
    assertThat(countLedgerTransactions(id)).isZero();                       // 원장 거래도 없다
}
```

> **함정: 자기 호출(self-invocation)에는 @Transactional 이 안 걸린다**
>
> 같은 클래스 안에서 `this.otherMethod()` 로 부르면 프록시를 거치지 않아 `otherMethod` 의 @Transactional(REQUIRES_NEW 등)이 무시된다. 그래서 LoginThrottle 은 AuthService 와 **다른 빈**이다. private 메서드에 붙인 @Transactional 도 같은 이유로 동작하지 않는다.

13 JPA 와 JdbcTemplate 섞어 쓰기

이 프로젝트는 둘 다 쓴다. 기준은 "엔티티의 생애(생성·규칙·변경)를 다루는가" 다.

JPA (엔티티)	JdbcTemplate (SQL 직접)
계좌·원장·이체·멱등성·고객·관리자: 규칙이 있고 상태가 바뀌는 것	관리자 요약(집계), 일 마감(대량 집계·INSERT … SELECT), 감사 로그 INSERT, 로그인 시도·횟수 제한(UPDATE 결과 건수가 필요한 카운터), 대사 테스트의 조작

둘은 **같은 트랜잭션, 같은 커넥션**을 쓴다. 스프링 부트가 만든 `JpaTransactionManager` 가 JDBC 커넥션을 공유해 주기 때문에, 한 서비스 메서드 안에서 섞어도 함께 커밋·롤백된다.

> **함정: JdbcTemplate 은 영속성 컨텍스트를 모른다**
>
> - JPA 로 고친 값이 아직 flush 전이면, 같은 트랜잭션의 JdbcTemplate SELECT 는 **고치기 전 값**을 본다(Hibernate 가 JDBC 호출 전에는 자동 flush 하지 않는다). 필요하면 먼저 flush 한다.
> - 반대로 JdbcTemplate 으로 UPDATE 한 행을 이미 JPA 가 1차 캐시에 들고 있으면, 그 엔티티는 **옛날 값**을 계속 보여 준다.
>
> 이 프로젝트는 같은 테이블을 한 트랜잭션에서 두 방식으로 고치지 않도록 나눠 두어 이 문제를 피했다.

14 설정: validate, open-in-view

`src/main/resources/application.properties`

```
# 테이블은 Flyway 가 만들고, JPA 는 엔티티와 테이블이 일치하는지 검사만 한다. 다르면 앱이 시작되지 않는다.
spring.jpa.hibernate.ddl-auto=validate
```

ddl-auto	동작	쓰는 곳
create / create-drop	시작할 때 테이블을 지우고 새로 만든다	학습용 예제. 운영 금지
update	모자란 열·테이블을 추가(지우지는 않음)	운영 금지 (이력이 남지 않고 예측 불가)
validate	엔티티와 테이블이 맞는지만 검사	**이 프로젝트**: 스키마는 Flyway SQL 이 책임
none	아무것도 안 함	

open-in-view (OSIV)

스프링 부트의 `spring.jpa.open-in-view` 기본값은 **true** 다. 요청이 시작될 때 영속성 컨텍스트를 열어 **응답을 다 쓸 때까지** 유지한다. 그래서 컨트롤러·JSON 변환 중에도 지연 로딩이 된다. 대신 그 요청 동안 DB 커넥션을 오래 쥘 수 있다(트랜잭션이 끝나도 커넥션을 반납하지 않는 경우가 생긴다). 이 프로젝트는 이 값을 따로 정하지 않아 기본값(true)이고, 시작 로그에 경고가 찍힌다.

> **생각해 보기**
>
> 이 프로젝트에는 지연 로딩 연관관계가 없고 응답은 DTO 로 바꿔서 내보낸다. 그렇다면 `open-in-view=false` 로 바꿔도 될까? 바꾸면 무엇이 좋아지나? → 실습 6

15 JPA 함정 모음

증상	원인	이 프로젝트의 대응
고쳤는데 저장이 안 된다	트랜잭션 밖(준영속)에서 고침	변경은 서비스 @Transactional 안에서만
save 안 했는데 UPDATE 가 나갔다	영속 엔티티의 변경 감지	setter 없음, 변경 메서드만
목록 하나에 쿼리 수십 개	N+1 (지연 로딩 반복)	연관관계 없음, IN 으로 모아 조회
LazyInitializationException	세션이 닫힌 뒤 지연 로딩 접근	연관관계 없음, DTO 변환
enum 값이 엉뚱하게 바뀜	ORDINAL 저장 후 enum 순서 변경	EnumType.STRING
id 가 50씩 건너뜀 / 중복 키	allocationSize 와 시퀀스 INCREMENT 불일치	둘 다 1
제약 위반이 엉뚱한 곳(커밋)에서 터짐	flush 가 커밋 때 일어남	필요한 곳은 saveAndFlush
REQUIRES_NEW 가 안 먹힘	자기 호출, private 메서드	별도 빈으로 분리
Set 에 넣은 엔티티를 못 찾음	바뀌는 값으로 equals/hashCode	id 기준 equals, 고정 hashCode
동시 요청에 잔액이 틀림	읽고-계산하고-쓰기 사이의 경쟁	비관적 락
JDBC 로 읽은 값이 JPA 와 다름	flush 전 / 1차 캐시	같은 테이블을 섞어 고치지 않음
로컬은 되는데 CI 에서만 시각이 1µs 다름	Linux 나노초 vs TIMESTAMP(6)	`DbTime.now()` 로 마이크로초 절삭

PART 3

도메인과 설계

은행 코드가 일반 CRUD 와 다른 지점들

16 복식부기 원장

잔액(account.balance)은 "지금 얼마인가" 를 빨리 보여 주는 **요약**이고, 진짜 기록은 원장이다. 모든 거래는 **차변(DEBIT) 합 = 대변(CREDIT) 합**이 되도록 두 줄 이상의 분개로 남는다.

거래	차변 (DEBIT)	대변 (CREDIT)	읽는 법
입금	현금 CASH	고객예금 CUSTOMER_DEPOSIT	은행 현금(자산) 증가, 고객에게 갚을 돈(부채) 증가
출금·해지 지급	고객예금	현금	부채 감소, 현금 감소
이체 (출금 쪽)	고객예금(보내는 계좌)	이체 가계정 TRANSFER_CLEARING	돈이 가계정을 거쳐 이동. 가계정은 들어온 만큼 나가 합이 0
이체 (입금 쪽)	이체 가계정	고객예금(받는 계좌)	

불변 조건 (일 마감이 매일 확인)

1. 계좌 잔액 = 그 계좌 고객예금 분개의 (대변 − 차변) 합
2. 거래마다 차변 합 = 대변 합
3. 이체 가계정 합 = 0, 이체 하나는 원장 거래 정확히 2건
4. 원장은 고치지 않는다. 틀리면 반대 거래(역분개)를 추가한다

거래내역 정렬은 `created_at` 이 아니라 **id** 로 한다. 같은 계좌의 거래는 잠금으로 한 줄로 처리되고 시퀀스는 처리 순서대로 번호를 주므로 id 순서가 곧 거래 순서다. 시각은 같은 값이 겹칠 수 있다.

17 멱등성 키

모바일에서 "입금" 을 눌렀는데 응답 직전에 네트워크가 끊겼다. 앱은 성공했는지 모른다. 다시 보내면 두 번 입금될 수 있다. 그래서 돈이 움직이는 요청은 `Idempotency-Key` (요청마다 새 UUID, 재시도면 같은 값)가 필수다.

상황	서버 동작
처음 보는 키	처리하고 (키, 요청 내용 지문, 결과 id) 를 같은 트랜잭션에 저장
같은 키 + 같은 내용	처리하지 않고 처음 결과를 그대로 201 로. 헤더 `Idempotent-Replayed: true`
같은 키 + 다른 내용	422 IDEMPOTENCY_KEY_REUSED (클라이언트 버그)

핵심: 잠금 → 키 확인 순서

키를 먼저 확인하고 잠그면, 같은 요청 두 개가 동시에 와서 둘 다 "처음 보는 키" 로 판단해 두 번 처리한다. **계좌를 먼저 잠그면** 두 번째 요청은 잠금에서 기다리다가, 첫 번째가 키 기록과 함께 커밋한 뒤에 키를 확인해 저장된 결과를 받는다. 서로 다른 계좌를 같은 키로 동시에 보내는 경우(잠금으로 줄 세울 수 없음)는 키 유니크 제약 + saveAndFlush 가 마지막에 막는다.

재전송 응답의 잔액은 "지금 잔액" 이 아니라 그때 원장에 남긴 `balance_after` 다. 처음 응답과 바이트까지 같아야 하기 때문이다 (그래서 시각도 DB 정밀도로 맞췄다, DbTime).

화면 쪽 짝: `useIdempotentMutation` 훅이 요청 내용(지문)이 같으면 같은 키를 재사용하고, 처리 중 중복 클릭을 막는다.

18 이체와 데드락

transfer/TransferService.java

```
Account first  = lock(Math.min(fromId, toId));   // 항상 id 작은 쪽부터
Account second = lock(Math.max(fromId, toId));
Account from = first.getId().equals(fromId) ? first : second;
Account to   = from == first ? second : first;
...
from.withdraw(amount);     // 해지·잔액 부족 검사
to.deposit(amount);        // 해지·한도 검사 — 실패하면 위 출금도 롤백
Transfer transfer = transferRepository.save(Transfer.of(fromId, toId, amount));
ledgerService.recordTransfer(transfer.getId(), fromId, toId, amount, from.getBalance(),
to.getBalance());
```

보내는 계좌부터 잠그면, A→B 와 B→A 가 동시에 올 때 서로 상대가 쥔 계좌를 기다리는 **순환 대기(데드락)**가 생긴다. Oracle 은 이를 감지해 한쪽을 ORA-00060 으로 실패시킨다. 잠그는 순서를 방향과 무관하게 고정하면 순환이 생길 수 없다. 데드락 예방의 일반 원칙: **여러 자원을 잠글 때는 모두가 같은 순서로**.

받는 사람은 계좌번호로 지정할 수 있다. 화면은 먼저 `GET /accounts/lookup` 으로 "홍*동" 처럼 가린 이름을 보여 주고 확인을 받는다. 이 조회는 남의 계좌 정보를 알아내는 데 쓰일 수 있어 고객당 시간당 30번으로 제한한다.

19 인증·인가 (JWT)

결정	이유
세션 대신 JWT, STATELESS	서버가 로그인 상태를 저장하지 않는다. 서버를 여러 대 띄워도 같다. 대신 만료 전 강제 무효화가 어려워 수명을 30분으로 짧게
HS256 서명	서버 하나가 발급·검증하므로 대칭키(JWT_SECRET)로 충분. 토큰은 **서명만** 되어 누구나 읽을 수 있으므로 비밀을 넣지 않는다
scope = ADMIN / CUSTOMER, cid	Spring Security 가 `SCOPE_ADMIN` 권한으로 바꿔 URL 규칙에 쓴다. 고객 번호(cid)는 서명돼 위조할 수 없다
CSRF 끔	토큰을 쿠키가 아니라 Authorization 헤더로 보내 브라우저가 자동으로 실어 보내지 않는다
남의 계좌는 404	403 이면 "그 번호의 계좌가 있다" 가 드러나 번호를 바꿔 가며 찾아낼 수 있다
본인 계좌 확인은 컨트롤러에서	"어느 계좌인가" 는 경로·본문을 봐야 알 수 있어 URL 규칙으로는 못 정한다 → `AccountAccessPolicy`
로그인 실패 5번 → 15분 잠금	비밀번호 확인 **전에** 실패로 미리 세고(REQUIRES_NEW, 행 잠금) 성공하면 지운다. 동시에 여러 번 시도해 횟수를 건너뛰는 것을 막는다. 없는 아이디도 똑같이 잠가 계정 존재를 숨긴다
없는 아이디도 해시 비교	응답 시간 차이로 아이디 존재를 알아내지 못하게(타이밍 공격)

auth/SecurityConfig.java (요약)

```
http.csrf(csrf -> csrf.disable())
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.POST, "/auth/login", "/auth/customer/signup",
"/auth/customer/login").permitAll()
        .requestMatchers("/admin/**").hasAuthority("SCOPE_ADMIN")
        .requestMatchers(HttpMethod.GET, "/accounts").hasAuthority("SCOPE_CUSTOMER")
        .requestMatchers("/accounts/**", "/transfers/**").hasAnyAuthority("SCOPE_ADMIN",
"SCOPE_CUSTOMER")
        .anyRequest().permitAll())                        // 없는 경로를 404 로 응답하려고
    .oauth2ResourceServer(o -> o.jwt(jwt -> {}));        // Bearer 토큰 검증
```

20 감사 로그와 횟수 제한

감사 로그는 "누가, 언제, 무엇을, 어느 계좌에, 결과는" 을 요청마다 남긴다. 성공뿐 아니라 실패·거부(401/403/잔액 부족)와 조회도 남긴다. 금액은 남기지 않는다(원장이 기준).

- **필터 위치**: 보안 필터 체인 안, 토큰 확인 **앞**. 앞이라서 무효 토큰 요청(401)도 남고, 체인 안이라서 요청이 끝날 때까지 SecurityContext(누가 요청했나)가 살아 있다.
- **업무 트랜잭션과 분리**: 응답 뒤(finally)에 따로 쓴다. 업무가 롤백돼도 "시도했다" 는 기록은 남는다.
- **빈으로 등록하지 않음**: 필터를 @Component 로 만들면 스프링 부트가 서블릿 필터로도 등록해 요청마다 두 번 기록된다. 그래서 SecurityConfig 에서 직접 new 해서 체인에 넣는다.

횟수 제한은 DB 의 고정 구간 카운터다. `UPDATE … SET hit_count = hit_count + 1` 이 행을 잠그므로 동시에 와도 하나씩 센다. 0건 갱신이면 그 구간의 첫 요청이라 INSERT 하고, 동시에 INSERT 가 겹치면 `DuplicateKeyException` 을 잡아 다시 UPDATE 한다("없으면 만들기" 를 DB 제약에 맡기는 패턴). 기록이 DB 에 있으니 서버가 여러 대여도 같은 기준으로 센다.

21 일 마감 배치 (Spring Batch)

매일 00:10 에 전날 거래를 집계(일계표)하고, 잔액과 원장이 맞는지 **대사**한다. 트리거나 권한을 우회한 변경도 여기서 찾아낸다.

Spring Batch 용어	뜻	이 프로젝트
Job / JobInstance / JobExecution	작업 정의 / (잡 + 파라미터) 한 묶음 / 실행 한 번	dailyClosingJob, 파라미터 closingId·businessDate
Step	잡을 이루는 단계, 순서대로	일계표 → 계좌 대사 → 원장 점검 → 마감 기록
Tasklet	한 번에 끝나는 단계	SQL 한 문장(INSERT … SELECT)으로 DB 안에서 집계
Chunk (Reader→Processor→Writer)	N개씩 읽고 처리하고 쓰고 커밋을 반복	계좌를 100개씩: 잔액·원장 합계 비교 → 불일치만 배치 INSERT
JobRepository	실행 이력 저장소 (BATCH_* 테이블)	Flyway V9 로 생성
@StepScope	실행마다 잡 파라미터로 새로 만드는 빈	businessDate 를 받는 accountSnapshotReader

- **JdbcPagingItemReader 의 sortKey 페이징**: OFFSET 이 아니라 "마지막으로 읽은 id 다음부터" 라 뒤쪽 페이지도 느려지지 않는다(키셋 페이징).
- **왜 계좌 대사만 청크인가**: 계좌 수만큼 커지는 유일한 단계라 한꺼번에 메모리에 올리지 않는다. 나머지는 DB 가 집계하는 것이 가장 빠르다.

- **트랜잭션**: 청크마다 커밋. 그래서 ClosingService.run 은 바깥 트랜잭션 없이 실행한다(12장).
- **같은 날짜 동시 실행 방지**: "조회해 보고 없으면 시작" 은 경쟁에 약하다. RUNNING 상태에 유니크 인덱스를 걸어 DB 가 하나만 통과시킨다.
- **오늘은 마감 거부**: 아직 거래가 생길 수 있어 결과가 확정되지 않는다. 날짜의 기준은 서버 시간대의 Clock(DB 의 SYSDATE 아님. 컨테이너는 UTC 일 수 있다).

22 DB 설계 (Oracle, Flyway, 권한)

코드 + DB 두 겹

규칙	코드	DB
잔액은 음수 불가	Account.withdraw 검사	`CHECK (balance >= 0)`
해지 계좌는 잔액 0, 해지 시각 있음	Account.close	`ck_account_closed`
해지 지급은 계좌당 1건	서비스 흐름	함수 기반 유니크 인덱스
멱등성 키 하나에 기록 하나	findCompleted	유니크 제약
원장·감사 기록은 추가만	updatable=false, 변경 메서드 없음	앱 계정 UPDATE/DELETE 권한 없음 + 트리거

코드의 검사는 친절한 에러 메시지를 위해, DB 제약은 **코드에 버그가 있어도** 마지막에 막기 위해 있다.

Flyway 마이그레이션

- `V1__…sql` ~ `V13__…sql`: 버전 순서대로 한 번씩. 이미 실행된 파일은 고치지 않는다(체크섬 오류). 바꾸려면 새 버전을 추가한다.
- `R__app_user_grants.sql`: 반복 실행. 파일이 바뀔 때마다 앱 계정 권한을 다 거두고 다시 준다.
- 서버는 마이그레이션하지 않고 **validate 만** 한다. 적용 안 된 파일이 있으면 시작을 거부한다. 마이그레이션은 별도 진입점(MigrationApplication)이 소유자 계정으로.

계정 분리 (최소 권한)

계정	할 수 있는 것
소유자 corebank	테이블·트리거 생성, 권한 부여. 마이그레이션에서만 쓴다
앱 corebank_app	원장·감사: SELECT·INSERT 만 / 계좌: balance·status·closed_at 열만 UPDATE / DDL 불가. 접속할 때 `ALTER SESSION SET CURRENT_SCHEMA` 로 소유자 테이블을 찾는다

원장 보호는 세 겹이다: **권한**(앱 계정은 못 고침) → **트리거**(소유자도 UPDATE/DELETE 거부) → **대사**(트리거를 끄고 고쳐도 다음 날 찾음).

시각

- `TIMESTAMP(6)` 은 마이크로초까지. Linux 의 나노초 시각을 그대로 저장하면 반올림돼 메모리 값과 1μs 어긋난다 → `DbTime.now()` 로 잘라서 만든다.
- "오늘" 은 앱의 Clock(서버 시간대) 기준. 컨테이너 이미지에 `TZ=Asia/Seoul`.

PART 4

품질과 운영

어떻게 믿을 수 있게 만들었나

23 테스트 전략

종류	도구	무엇을	예
단위	JUnit, AssertJ	엔티티 규칙, 순수 함수	AccountTest, NameMaskingTest, DateRange
웹 계층	`@WebMvcTest` + MockMvc	HTTP 변환, 검증, 상태 코드, 보안 규칙. 서비스는 가짜	TransferControllerTest
통합	`@SpringBootTest` + Testcontainers Oracle	실제 DB 의 트랜잭션·락·제약·트리거·권한	WithdrawalIntegrationTest, ClosingIntegrationTest
API 회귀	Postman CLI	실제 서버를 처음부터 끝까지 (171 요청, 374 검증)	postman/corebank-api···json
CI	GitHub Actions	push 마다 Linux 에서 전부	.github/workflows/ci.yml

- **H2 같은 메모리 DB 를 쓰지 않은 이유**: FOR UPDATE, 시퀀스, CHECK 제약, 트리거, 권한 오류 코드가 Oracle 과 다르다. 은행 코드에서 가장 중요한 부분이 바로 이것들이라 실제 Oracle 을 Docker 로 띄운다.
- **컨테이너는 static 하나**: 스프링은 설정이 다른 테스트 클래스마다 컨텍스트를 새로 만든다. 컨텍스트마다 Oracle 을 띄우면 자원이 모자라 느려진다.
- **테스트끼리 DB 를 공유**한다. 그래서 각 테스트는 자기가 만든 계좌만 보고, 날짜를 바꿨으면 `@AfterEach` 로 되돌린다.
- **동시성 테스트**: ExecutorService 로 진짜 스레드를 띄워 동시에 보내고, 결과 건수와 원장 합계를 SQL 로 확인한다.
- **스파이로 실패 주입**: `@MockitoSpyBean` 으로 진짜 빈의 한 메서드만 실패시켜 롤백을 확인한다(12장).
- **권한도 테스트**: 테스트의 앱도 앱 계정으로 돈다. 권한이 모자라면 테스트가 깨지고, 새 테이블에 권한을 빠뜨리면 검사 테스트가 깨진다.

24 배포 자동화

git push main	GitHub Actions `test` 작업: Testcontainers Oracle 로 전체 테스트
image 작업	테스트 통과 시 bootJar → Docker 이미지(amd64·arm64) → GHCR 에 커밋 sha 와 latest 태그
deploy.sh ①	이미지 받기 → `migrate` 컨테이너(소유자 계정)로 마이그레이션. 실패하면 서버는 그대로 두고 멈춤
deploy.sh ②	`app` 컨테이너 교체(앱 계정만) → `/actuator/health` 가 90초 안에 UP 인지 → 아니면 이전 이미지로 되돌림

- **한 이미지, 두 실행**: 기본은 서버, `migrate` 인자면 마이그레이션. 테스트를 통과한 바로 그 jar 를 복사만 한다.
- **되돌리기가 되는 이유**: 서버의 validate 는 자기보다 새 마이그레이션은 허용한다. 대신 마이그레이션은 이전 코드와 호환되게(열 삭제는 다음 배포로) 쓴다.
- 비루트 사용자, 시간대 Asia/Seoul, health 외의 actuator 는 닫음.

25 화면(React)과의 약속

프론트(`project-corebank-front`)는 React + TypeScript + TanStack Query. 개발 서버가 `/api` 를 백엔드로 넘겨 CORS 설정이 필요 없다. 백엔드 공부 관점에서 볼 곳은 세 군데다.

- `api/client.ts` : 토큰을 Authorization 헤더에 붙이고, 공통 에러 형식 `{code, message}` 를 해석한다. 401 이면 로그인으로.
- `hooks/useIdempotentMutation.ts` : 멱등성 키를 만들고, 결과를 모르는 실패 뒤 같은 내용이면 같은 키로 재시도한다. 중복 클릭은 ref 로 즉시 막는다.
- `components/RequireAuth.tsx` : 역할별 화면 보호. 단, 진짜 보호는 서버(403/404)이고 화면은 편의다.

PART 5

스스로 점검

소리 내어 답해 보고, 막히면 해당 장으로 돌아가기

26 면접형 질문 40

태그: JPA 트랜잭션 DB·락·배치 설계·테스트 · 답을 가리고 먼저 말해 본 뒤 확인한다.

JPA Q1. JPA, Hibernate, Spring Data JPA 의 차이는?

JPA 는 표준 명세, Hibernate 는 그 구현체(SQL 생성·변경 감지), Spring Data JPA 는 리포지토리 인터페이스만으로 구현을 만들어 주는 편의 계층.

JPA Q2. 영속성 컨텍스트란? 이점 세 가지는?

트랜잭션 동안 엔티티를 보관하는 공간. 1차 캐시(같은 id 는 같은 객체), 변경 감지(자동 UPDATE), 쓰기 지연(SQL 을 모아 flush 때 실행).

JPA Q3. 입금 코드에 save() 가 없는데 잔액이 저장되는 이유는?

잠금 조회로 읽은 Account 는 영속 상태다. 필드를 바꾸면 flush 때 스냅숏과 비교해 Hibernate 가 UPDATE 를 만든다.

JPA Q4. 준영속 엔티티를 고치면 어떻게 되나?

아무 일도 안 일어난다. 반영하려면 트랜잭션 안에서 다시 조회해 고치거나 merge 해야 한다. 이 프로젝트는 항상 서비스 트랜잭션 안에서 조회 후 변경.

JPA Q5. flush 는 언제 일어나나? 커밋과 다른 점은?

커밋 직전, (AUTO 모드에서) 영향받는 JPQL/네이티브 쿼리 직전, 직접 호출할 때. flush 는 SQL 을 DB 로 보내는 것일 뿐 커밋이 아니라 롤백되면 사라진다.

JPA Q6. 멱등성 기록을 saveAndFlush 로 하는 이유는?

유니크 제약 위반을 try 안에서 바로 터뜨려 업무 예외(키 재사용)로 바꾸기 위해. save 만 하면 커밋 때 터져 잡을 수 없다.

JPA Q7. save() 는 언제 persist 이고 언제 merge 인가?

Spring Data 는 새 엔티티(기본: id 가 null)면 persist, 아니면 merge.

JPA Q8. SEQUENCE 와 IDENTITY 전략 차이는?

SEQUENCE 는 INSERT 전에 id 를 받아 INSERT 를 미루고 배치로 묶을 수 있다. IDENTITY 는 INSERT 해야 id 를 알아 persist 즉시 INSERT, JDBC 배치 불가.

JPA Q9. allocationSize 는 무엇이고 왜 1로 했나?

시퀀스 한 번에 미리 받아 둘 번호 수(기본 50). DB 시퀀스 INCREMENT BY 와 같아야 한다. V1 의 시퀀스가 1씩 증가라 1. 대신 저장마다 NEXTVAL 왕복.

JPA Q10. EnumType.ORDINAL 이 위험한 이유는?

순번을 저장하므로 enum 중간에 값을 넣거나 순서를 바꾸면 기존 데이터의 의미가 바뀐다.

JPA Q11. 엔티티에 protected 기본 생성자가 필요한 이유는?

JPA 가 조회 결과로 객체를 만들 때 리플렉션으로 기본 생성자를 쓰고, 프록시가 상속해야 하므로. public 이 아니라 앱 코드는 못 쓴다.

JPA Q12. updatable = false 는 무엇을 하나? 여기서 왜 중요해졌나?

그 열을 UPDATE 문에서 뺀다. 앱 DB 계정이 balance/status/closed_at 열만 UPDATE 권한이 있어, 나머지 열이 UPDATE 문에 들어가면 권한 오류가 난다.

JPA Q13. N+1 문제란? 해결법은?

목록 1번 + 건마다 1번 쿼리. IN 으로 모아 조회, fetch join, batch fetch size. EAGER 는 해법이 아님.

JPA Q14. 이 프로젝트가 @ManyToOne 을 안 쓴 이유와 단점은?

원장은 기록이라 id 값으로 충분, 쿼리 예측성과 패키지 결합 감소. 단점: 객체 그래프 탐색 불가, 필요하면 직접 조회.

JPA Q15. LazyInitializationException 은 언제 나나?

영속성 컨텍스트가 닫힌 뒤 지연 로딩 프록시를 처음 건드릴 때. OSIV 가 꺼져 있으면 컨트롤러·JSON 변환에서 자주 난다.

JPA Q16. open-in-view 란? 기본값과 장단점은?

요청 끝까지 영속성 컨텍스트를 유지. 부트 기본 true. 지연 로딩이 뷰에서도 되지만 커넥션을 오래 쥘 수 있다.

JPA Q17. readOnly = true 의 효과는?

Hibernate 세션 읽기 전용 + flush 모드 MANUAL: 스냅숏·변경 감지 생략, 실수로 고쳐도 UPDATE 안 나감.

JPA Q18. 엔티티 equals/hashCode 를 어떻게 만들었고 왜?

id 로만 비교(저장 전엔 자기 자신만), hashCode 는 클래스 고정값. 바뀌는 값 비교·해시 변화·프록시 문제를 피하려고.

JPA Q19. JPQL 과 SQL 의 차이는?

JPQL 은 엔티티·필드 이름 기준이고 Hibernate 가 DB SQL 로 번역한다. DB 전용 문법은 네이티브 쿼리로.

JPA Q20. Page 를 반환하면 쿼리가 몇 번 나가나?

목록 + COUNT 두 번. 커서 방식은 COUNT 가 필요 없다.

TX Q21. @Transactional 이 동작하는 원리는?

스프링 AOP 프록시가 메서드 앞뒤에서 트랜잭션을 시작·커밋·롤백한다.

TX Q22. 자기 호출 시 @Transactional 이 무시되는 이유는?

this 로 부르면 프록시를 거치지 않는다. 그래서 별도 빈으로 분리한다(LoginThrottle).

TX Q23. 기본 롤백 규칙은?

RuntimeException·Error 는 롤백, 체크 예외는 커밋(rollbackFor 로 변경 가능).

TX Q24. MANDATORY 를 원장 서비스에 쓴 이유는?

원장 기록이 업무 트랜잭션 밖에서 따로 커밋되는 실수를 코드 차원에서 막으려고. 트랜잭션 없이 부르면 예외.

TX Q25. REQUIRES_NEW 를 로그인 시도 카운트에 쓴 이유는?

로그인이 예외로 끝나 롤백돼도 실패 횟수는 남아야 하므로 따로 즉시 커밋.

TX Q26. JPA 와 JdbcTemplate 을 같은 트랜잭션에서 쓸 때 주의점은?

같은 커넥션·트랜잭션이지만, JDBC 는 flush 전 변경을 못 보고 JPA 1차 캐시는 JDBC 변경을 모른다.

락 Q27. 갱신 분실(lost update)을 설명하라.

두 트랜잭션이 같은 값을 읽고 각자 계산해 쓰면 한쪽 변경이 사라진다. 잔액 검사도 읽은 값 기준이라 둘 다 통과한다.

락 Q28. 비관적 락과 낙관적 락의 차이와 선택 기준은?

먼저 잠그고 기다림 vs 버전 비교 후 실패·재시도. 충돌 잦고 실패하면 안 되는 계좌 거래는 비관적.

락 Q29. 이체에서 데드락을 어떻게 막았나?

두 계좌를 항상 id 작은 쪽부터 잠가 순환 대기를 없앴다.

락 Q30. 멱등성 키 확인을 잠금 뒤에 하는 이유는?

잠금 전에 확인하면 동시에 온 같은 요청이 둘 다 처음으로 판단한다. 잠금 뒤면 두 번째는 첫 번째 커밋 후 기록을 본다.

설계 Q31. 잔액이 있는데 원장을 따로 두는 이유는?

잔액은 빠른 조회용 요약, 원장은 변경 불가 기록. 둘이 맞는지 매일 대사해 오류·조작을 찾는다.

설계 Q32. 원장이 틀렸으면 어떻게 고치나?

UPDATE 하지 않고 반대 거래(역분개)를 추가한다.

설계 Q33. 코드 검사가 있는데 DB 제약을 또 두는 이유는?

코드 버그나 직접 SQL 에도 마지막에 막기 위해. 코드 검사는 친절한 메시지용.

설계 Q34. 남의 계좌 접근에 403 이 아니라 404 를 주는 이유는?

계좌 존재 여부가 드러나 열거 공격에 쓰이지 않게.

설계 Q35. JWT 를 강제로 무효화하기 어려운 이유와 대응은?

서버가 상태를 저장하지 않으므로. 수명을 짧게(30분), 필요하면 jti 차단 목록·리프레시 토큰.

배치 Q36. Spring Batch 의 tasklet 과 chunk 차이는?

한 번에 끝나는 단계 vs N개씩 읽기·처리·쓰기·커밋 반복. 계좌 수만큼 커지는 대사만 chunk.

배치 Q37. OFFSET 페이징 대신 키셋 페이징을 쓰는 이유는?

OFFSET 은 뒤쪽일수록 앞을 다 건너뛰느라 느리다. '마지막 id 다음부터' 는 인덱스로 바로 간다.

DB Q38. ddl-auto=validate 와 Flyway 의 역할 분담은?

Flyway 가 스키마를 만들고 이력을 관리, Hibernate 는 엔티티와 맞는지만 검사.

DB Q39. DB 계정을 소유자/앱으로 나눈 이유는?

앱 계정이 탈취돼도 트리거를 끄거나 원장을 지우는 DDL·DML 을 못 하게(최소 권한).

테스트 Q40. H2 대신 Testcontainers Oracle 을 쓴 이유는?

락·시퀀스·제약·트리거·권한·오류 코드가 실제 DB 와 같아야 은행 로직을 검증할 수 있다.

27 손으로 해 보는 실습 12

모든 실습은 브랜치를 만들어서 하고(`git switch -c study`), 끝나면 `git switch main` 으로 돌아간다. 테스트 실행: `./gradlew test --tests '*WithdrawalIntegration*'` 처럼 하나씩. Docker(Colima)가 켜져 있어야 한다.

#	해 볼 것	관찰할 것 / 배우는 것
1	`application.properties` 에 `spring.jpa.show-sql=true`, `spring.jpa.properties.hibernate.format_sql=true` 를 넣고 입금 테스트 하나 실행	FOR UPDATE, NEXTVAL, INSERT, UPDATE 의 **실제 순서**. UPDATE 가 어느 시점에 나가는지(6, 7장)
2	`findByIdForUpdate` 의 `@Lock` 을 지우고 `concurrentWithdrawalsNeverOverdraw` 실행	성공 건수가 3보다 많아지거나 잔액이 틀어지는 것(갱신 분실, 10장). 여러 번 돌려 보기
3	TransferService 에서 잠금 순서를 "보내는 계좌 먼저" 로 바꾸고, A→B 와 B→A 를 동시에 보내는 테스트를 직접 작성	ORA-00060 데드락(18장)
4	`IdempotencyService.record` 의 `saveAndFlush` 를 `save` 로	멱등성 테스트 중 무엇이 깨지고 예외가 어디서 나는지(7장)
5	`Account.ownerName` 의 `updatable = false` 를 지우고 입금 테스트	ORA-41900 권한 오류: Hibernate 가 모든 열을 UPDATE 한다는 증거(5, 6장). 그다음 `@DynamicUpdate` 를 붙이면?
6	`spring.jpa.open-in-view=false` 추가 후 전체 테스트	통과하는지, 시작 로그의 경고가 사라지는지(14장). 왜 통과하는지 설명하기
7	Account 에 `@Version` 을 추가하는 마이그레이션(V14)과 함께 낙관적 락 버전의 출금을 만들어 보기	동시 출금 시 ObjectOptimisticLockingFailureException, 재시도 로직이 필요해지는 것(10장)
8	LedgerTransaction 에 `@ManyToOne(fetch = LAZY) Account account` 를 두는 가지를 만들고, 거래내역에서 `tx.getAccount().getOwnerName()` 을 반복	SQL 로그에서 N+1 확인 → fetch join 으로 고치기(9장)
9	`AuthService` 안에 `@Transactional(propagation = REQUIRES_NEW)` private 메서드를 만들어 실패 카운트를 옮겨 보기	로그인 실패가 세지지 않는 것: 자기 호출과 private 의 함정(12장)
10	서비스 메서드에서 `account.deposit()` 직후 JdbcTemplate 으로 그 계좌의 balance 를 SELECT	JDBC 는 flush 전 값을 본다(13장). `entityManager.flush()` 를 넣으면?
11	정정(역분개) API 를 설계·구현: 관리자가 거래 id 를 주면 반대 분개를 추가하고 잔액을 되돌림	원장 불변·멱등성·잠금·감사 로그·권한을 한 기능에 모두 적용해 보기(16장). 이 프로젝트의 다음 후보이기도 하다
12	새 테이블을 하나 만들고 `R__app_user_grants.sql` 에 권한을 빼먹은 채 테스트	어떤 테스트가 어떤 메시지로 잡아 주는지(22장)

28 추천 공부 순서

순서	읽을 것	할 것
1일	1~3장, Account·AccountService	입금 흐름(2장)을 종이에 그리고 말로 설명하기
2일	4~7장	실습 1, 4, 5. SQL 로그를 보며 영속성 컨텍스트 타임라인 다시 그리기
3일	8~9장, 리포지토리 전부	실습 8 (N+1 만들고 고치기)
4일	10~12장	실습 2, 3, 9. 동시성 테스트 코드를 직접 다시 써 보기
5일	13~15장, 16~18장	실습 6, 10. 질문 Q1~Q30 에 답하기
6일	19~22장	실습 12. 보안 규칙과 권한 표를 보지 않고 다시 써 보기
7일~	23~25장	실습 7, 11 (가장 크다). 질문 40개 전체 다시

마지막으로

AI 가 코드를 써 줘도, **왜 그 줄이 있어야 하는지**는 내가 설명할 수 있어야 한다. 좋은 연습법: 아무 메서드나 하나 골라 줄 하나를 지웠을 때 무엇이 깨지는지 먼저 예측하고, 테스트로 확인한다. 예측이 맞으면 이해한 것이고, 틀리면 그곳이 공부할 곳이다.
